API Spec: ROI Scoring Engine

TL;DR

A backend computation engine that reads from the PI enrichment cache and Trac hours database to produce two scores per REDCap request: (1) Weighted Return Score — a 0–100 composite of four return categories (Publication Impact, Journal Prestige, Funding Activity, Research Output) using team-configurable category weights; (2) Effort-Adjusted ROI Score — the Weighted Return Score divided by an effort multiplier derived from Trac hours, rescaled to 0–100. The engine runs on every REDCap upload against cached enrichment data, stores all score components in an ML-ready schema, and never makes live API calls. Internal metric weights within each category are fixed in v1.

Goals

Business Goals

- Generate mathematically robust, auditable ROI scores for every research request in the system.
- Enable team leads to reflect current organizational priorities in request ranking through configurable category weights.
- Maintain traceable, ML-ready data—storing all scoring inputs, intermediate values, outputs, and weight history.
- Ensure scoring is fully automated, repeatable, and fast (<2 min per full queue).
- Support data-driven queue and resource management, preparing for future ML model integration.

User Goals

- Give data managers clear, side-by-side Weighted Return and Effort-Adjusted ROI scores per request in the queue.
- Allow team leads to adjust category weights via intuitive UI sliders and preview queue impact before committing.
- Provide transparent subscores and drilling to all input metrics for each record.
- Flag and explain outlier/missing data with actionable status indicators (COMPLETE, PARTIAL, FAILED).
- Empower users to trust and understand every displayed score and its breakdown.

Non-Goals

- The engine does **not** make final triage or processing decisions; it only informs human judgment.
- It does **not** fetch live API data; all enrichment data is consumed from internal caches.

- Metric-level weight adjustment is not included in v1 (category-level only, internal metric weights are fixed).
- No ML prediction logic is run in v1; it purely produces the training-ready data for v2 ML work.

User Stories

- **Backend System**
- As the backend, I want to read all PI enrichment metrics from the cache and compute normalized subscores for each category, so all scoring is fast, repeatable, and API-independent.
- As the backend, I want to apply configurable category weights to produce a composite return score, divide by the effort multiplier, and dynamically rescale to deliver both scores on the same 0–100 scale.
- As the backend, I want to store each raw metric, normalized score, category subscore, applied weight, and final output in separate, queryable columns for every scoring event, enabling ML-readiness.
- **Data Manager**
- As a data manager, I want to see both weighted and effort-adjusted scores as sortable columns, so I can distinguish raw potential from investment-adjusted value.
- As a data manager, I want clear, color-coded flags for requests with missing or partial data, enabling manual review and override.
- **Team Lead**
- As a team lead, I want to adjust category weights using sliders and instantly preview how queue order would change, so I can align ranking with current priorities.
- As a team lead, I want to save/load named weight presets and see an audit log of past configurations for transparency and best practice sharing.

Functional Requirements

- **Inputs (Priority: High)**
- Reads from `pi_enrichment_cache`: mean_rcr, avg_author_count, impact_factor_avg, jcr_quartile_score, h5_index_avg, h5_median_avg, active_grant_count, total_award_amount, follow_on_grant (boolean), publication_volume_5yr, collab_network_size, niche_journal_bonus_eligible (boolean), enrichment_status.
- From `request_publications`: COUNT(DISTINCT pmid) for each PI.
- From Trac: total_hours per request (sum over all tickets).
- From `weight_config`: active weights (`w1, w2, w3, w4`) for each of four scoring categories.
- From `admin_config`: current niche_bonus_multiplier (default 1.125), team average total_hours for baseline effort adjustment.

- **Scoring Tier Classification (Priority: High)**

Before computing category subscores, the engine classifies each request into one of three tiers based on data availability.

Tier 1 — Full Enrichment: PI enrichment cache is COMPLETE or PARTIAL with sufficient data. Standard scoring pipeline applies with no special labeling beyond the existing enrichment status flag.

Tier 2 — Proxy Scoring (Enrichment Pending): PI enrichment cache is empty (new PI never enriched) or FAILED for all sources. REDCap proxy signals substitute for missing subscores. Faculty Rank proxy (0–100 scale): Professor=100, Associate Professor=75, Assistant Professor=50, Instructor=25, Resident/Fellow=10, NULL=no proxy available. Funding Source proxy (0–100 scale): Federal=100, Industry=75, Foundation=50, Internal=25, None/Unknown=0, NULL=no proxy available. Faculty Rank proxy substitutes for Publication Impact and Research Output subscores when those categories have NULL enrichment data. Funding Source proxy substitutes for Funding Activity subscore when NULL. Journal Prestige subscore cannot be proxied from REDCap — it remains NULL when enrichment is unavailable. Proxy confidence discount: all proxy-based subscores multiplied by proxy_confidence_discount (default 0.85, stored in admin_config, never hardcoded) before entering the Weighted Return Score formula. This ensures real enrichment scores always dominate proxy estimates when comparing PIs. Partial proxy: if only one proxy is available, it contributes its discounted weight — the missing proxy contributes nothing, not zero. UI label: amber flag 'Score estimated from REDCap data — enrichment pending.' Each proxy subscore bar in the breakdown is labeled 'Estimated.' Pre-discount and post-discount proxy subscore values are stored separately in the scoring event.

Tier 3 — Insufficient Data: Both enrichment data AND REDCap proxies are unavailable — faculty rank IS NULL AND funding source IS NULL AND all enrichment failed. All subscores are NULL. UI flag: red indicator 'Insufficient data for scoring.' Excluded from Open Requests Export. Appears in Full Queue Export with NULL score columns. Ranked last within the unscored group, ordered by Date of Request ascending (most recent first). Data manager can manually override rank with a note. This tier represents unknown impact — not low impact. NULL is never treated as zero.

- **Normalization (Priority: High)**
- All raw input metrics normalized via min-max scaling over all PIs in `pi_enrichment_cache`:
  `normalized = ((value - min) / (max - min)) * 100`
- NULL = missing, not zero. Category scores are computed from present values only (flagged PARTIAL if any input is NULL).
- **Category Subscore Computation (Priority: High)**
- Metric-to-category mapping and fixed internal weights:
  - *Publication Impact*: (0.70 × norm_mean_rcr) + (0.30 × norm_avg_author_count). NULL mean_rcr = subscore 0.
  - *Journal Prestige* (pre-niche): (0.30 × norm_impact_factor) + (0.30 × jcr_quartile_score) + (0.20 × norm_h5_index) + (0.20 × norm_h5_median). H5 NULL: redistribute 40% each to IF, JCR.

    - *Niche Bonus*: if `niche_journal_bonus_eligible = TRUE`, multiply Journal Prestige by `niche_bonus_multiplier` (default 1.125, range 1.10–1.15). Post-bonus value may exceed 100 and is not capped until queue-level rescaling.
    - *Funding Activity*: (0.50 × norm_total_award_amount) + (0.30 × norm_active_grant_count) + (0.20 × follow_on_grant_score). `follow_on_grant_score`: verified TRUE=100, unverified TRUE=50, else 0.
    - *Research Output*: (0.50 × norm_publication_volume_5yr) + (0.50 × norm_collab_network_size).
- When a category subscore is computed from proxy data (Tier 2), multiply by proxy_confidence_discount from admin_config (default 0.85) before it enters the Weighted Return Score. Store pre-discount and post-discount values in the scoring event for auditability.
- **Weighted Return Score (Priority: High)**
- Calculated as: `(w1 × Publication Impact) + (w2 × Journal Prestige) + (w3 × Funding Activity) + (w4 × Research Output)`
- Weights sum to 1.0. Default: 0.25 each. Weights are never hardcoded; always read from current active `weight_config`.
- **Effort-Adjusted ROI Score (Priority: High)**
- Effort multiplier = request_total_hours / team_average_total_hours (recomputed after every Trac upload).
- New requests with no history use historical avg for request type; fallback default is 2 hours.
- Effort-Adjusted Score = Weighted Return Score divided by Effort Multiplier.
- **Dynamic queue-level rescaling:** after effort division, min-max rescale Effort-Adjusted scores over the queue to 0–100, preserving order.
- **Score Storage (Priority: High)**
- Store all raw input metrics, normalized scores, category scores, niche bonus usage, weights, pre/post-effort scores, effort multiplier, enrichment status, and scoring_version as separate fields per request per scoring event in `request_scores`. No serialized blobs.
- Store the following additional fields to support scoring tier and proxy audit:
    - **scoring_tier** (VARCHAR: TIER_1/TIER_2/TIER_3)
    - **proxy_confidence_discount_used** (FLOAT, NULL if Tier 1)
    - **faculty_rank_proxy_score** (FLOAT, NULL if not used)
    - **funding_source_proxy_score** (FLOAT, NULL if not used)
    - **proxy_fields_used** (VARCHAR, comma-delimited list of proxies applied, e.g. 'faculty_rank,funding_source')
- **Weight Configuration (Priority: High)**
- `weight_config` table stores all config versions: (config_id, w1, w2, w3, w4, created_by, created_at, is_active, preset_name).
- Changing weights (Apply action) inserts a new row, deactivates previous, and triggers full queue rescore; audit log is retained.

- **Partial/Missing Data Handling (Priority: High)**
- NULL values propagate and are flagged as PARTIAL; no conversion to zero.
- Requests with FAILED enrichment (all data missing) are unscored, flagged in red, and excluded from ranking.
- Data manager can manually override order for PARTIAL/FAILED requests.
- **Rescoring Triggers (Priority: High)**
- Rescore queue on: REDCap upload, Trac upload, Apply new weights, or updated enrichment cache. Each rescore yields a new scoring event; past scores are never overwritten.

User Experience

Entry Point & First-Time User Experience

- Scores first surface to users upon uploading a REDCap export or upon a queue update.
- First upload triggers an automatic scoring run; UI shows both scores as columns with tooltips describing each.
- No complex onboarding needed: queue presentation is self-explanatory, with drilldowns and tooltips for transparency.

Core Experience

- **Step 1:** Data manager uploads or imports new REDCap export.
    - Engine reads `pi_enrichment_cache`, Trac hours, and current weights.
    - Score computation is automatic; results are available within 2 minutes.
- **Step 2:** Main queue displays two columns per request: Weighted Return Score and Effort-Adjusted ROI Score.
    - Default sort is by Effort-Adjusted; users may toggle sort by Weighted Return.
    - Tooltips on columns:
        - *Weighted Return Score*: "Raw research impact before effort adjustment."
        - *Effort-Adjusted ROI*: "Return score divided by hours invested—higher means better per-hour payoff."
- **Step 3:** Clicking a request expands the full breakdown panel:
    - Four category bars (0–100), raw input values, niche bonus flag, effort multiplier, subtotal calculations.
    - Status label: COMPLETE (all metrics), PARTIAL (some missing), or FAILED (all missing)—shown as green, amber, red.
    - For PARTIAL/FAILED: Data manager can leave manual rank note.
- **Step 4:** Weight Configuration Panel
    - Four sliders (min 5% each), live percentage display, auto-normalization.
    - Preview mode shows tentative queue reordering.
    - Apply triggers full rescore; preset save/load and audit log available.

- **Step 5:** Missing data handling in UI:
    - NULL scores as grey bars labeled "Data unavailable."
    - PARTIAL subscores flagged amber, FAILED flagged red, with dialogs explaining which data source(s) were unavailable.
    - If in Tier 2 (proxy): UI displays amber flag "Score estimated from REDCap data — enrichment pending." Proxy subscores are labeled "Estimated."
    - If in Tier 3 (insufficient data): Red "Insufficient data for scoring" flag appears, and request is excluded from Open Requests Export; shown in Full Queue Export with all score columns NULL.

Advanced Features & Edge Cases

- If Google Scholar data is missing, Journal Prestige redistributes weights and flags PARTIAL.
- Requests with zero effort hours (imputed default if missing) are flagged to avoid divide-by-zero.
- Out-of-bounds metric values (e.g., negative grant dollars) cause row-level error and warning in admin log.
- Queue-level min-max rescaling ensures ROI score is always 0–100, even if some raw values >100 after bonuses.
- All overrides, configuration changes, and manual data manager notes are logged with timestamp and user.

UI/UX Highlights

- Two-score columns with clear, concise tooltips.
- Expandable drilldown for transparent metric and subscore display.
- Color-coded statuses (COMPLETE—green, PARTIAL—amber, FAILED—red), with accessibility-compliant contrast.
- Responsive, sortable queue layout, optimized for desktop and tablet resolution.
- Sliders in the weight configuration panel have min/max enforcement, auto-normalization, and input validation.

Narrative

A data manager logs in to upload the latest REDCap dataset. The moment the records are ingested, the ROI Scoring Engine immediately pulls all available enrichment data and Trac hours, normalizes every relevant metric, and computes four category-specific subscores for each project. Using the currently active weights—set by the team lead to prioritize Funding Activity at 40% this grant cycle—the engine combines these into a weighted return score, then divides by the relative effort to generate an Effort-Adjusted ROI. Within seconds, the queue visibly updates: established, high-impact PIs with major grants rise to the top, while newer, lower-output projects settle further down. One request is flagged amber—some Google Scholar metrics were inaccessible—alerting the manager to needed review. The team lead opens the weight configurator, shifts Research Output down, and sees the queue instantly preview the new order. Satisfied, they apply the change and label it "Spring Grants Push," instantly archiving the decision. All scores, inputs, and breakdowns are recorded for full transparency, and the data is

ready for future predictive modeling—helping both users and leadership make sharper, data-driven decisions.

Success Metrics

- **Technical metrics**
    - Scoring latency: Rescore of 50 requests completes in under 2 minutes.
    - Score reproducibility: Identical inputs and weights always yield identical scores.
    - Zero silent failures: Every request has a logged scoring event (no NULL final status).
    - ML schema readiness: All fields are stored as discrete, queryable columns (no blobs).

User-Centric Metrics

- % of users reporting that the score breakdown is "clear and understandable" in post-launch survey.
- % of requests with manual override notes versus total—target <10% after 6 months (indicates trust).
- Preset usage: how often team leads adjust/save/load weight configurations.

Business Metrics

- Reduction in time spent on manual queue ranking (>25% post-launch).
- Increase in high-impact grants/publications among top-ranked, processed requests (>15% within 12 months).
- User satisfaction: ≥4/5 survey score from data managers and team leads.

Technical Metrics

- <2 minute full-queue rescore
- 99% uptime on score display and config panel
- No more than 0.5% scoring error rate (due to missing or corrupted enrichment fields)

Tracking Plan

- Per scoring event: record_id, all normalized subscores, category scores, weights, final scores, effort multiplier, enrichment & status, timestamp, scoring version.
- Per weight change: previous weights, new weights, user, timestamp, optional preset name.
- Alerts if any request returns NULL after a rescoring trigger.
- Queue and audit log events tracked for review.

Technical Considerations

Technical Needs

- Pulls from pi_enrichment_cache, request_publications, Trac hours, weight_config, admin_config.
- Implements queue-level normalization, subscore aggregation, effort divider, and rescaling.
- Outputs to request_scores with all discrete, queryable columns.
- Maintains weight_config with full history and is_active flag; triggers rescore logic on weight updates.

Integration Points

- REDCap for request records.
- Trac for team hours.
- pi_enrichment_cache fed by upstream APIs.
- Admin UI for config and audit display.

Data Storage & Privacy

- All scoring fields written as discrete columns (no JSON blobs).
- Full audit trail for both scores and weight history.
- Compliance with institutional data retention and access controls.
- No PHI, but stores PI names, IRB numbers, and grant info—subject to internal privacy and audit rules.

Scalability & Performance

- Expected queue: 20–200 requests; scoring latency tested up to 500 records.
- Dynamic min-max normalization recalculated at each run.
- Robust to partial/missing inputs and spikes in request volume.

Deployment Sequence

The following sequence must be strictly followed at deployment to ensure all historical and current requests are fully scored from day one.
Step 1 — Run the full PI enrichment cycle for all PIs found in the REDCap historical dataset (all records, open and closed). This populates pi_enrichment_cache before any scoring runs.
Step 2 — Run the PubMed historical backfill script for all closed requests (date_complete IS NOT NULL). This populates request_publications with historical PMIDs, which feeds iCite RCR and Research Output metrics used in scoring.
Step 3 — Run the scoring engine against all requests (historical and current) using the freshly populated enrichment cache. All requests receive a Tier 1 (enrichment), Tier 2 (proxy), or Tier 3 (insufficient data) score. This sequence ensures no request starts with an artificially empty or zero score due to deployment timing.

Potential Challenges

- Queue-level rescaling causes all scores to shift when new requests are added—users must understand scores are queue-relative.
- Null/zero discipline: NULL ≠ 0, must be maintained throughout pipeline.
- Historical benchmarking needed for normalization in early deployment (if <20 PIs).
- Dynamic fields and versioning; always display and store latest scores/styles.
- Weight preview is always in-memory and never writes to DB until Apply.
- Proxy scoring introduces a second scoring pathway that must be tested independently. QA must verify: (a) proxy scores are correctly discounted by proxy_confidence_discount; (b) a Tier 2 proxy score never outranks a Tier 1 enrichment score of equal underlying value; (c) NULL faculty rank and NULL funding source both correctly fall through to Tier 3, not scored as zero.

Milestones & Sequencing

Project Estimate

Medium: 5–7 days

Team Size & Composition

- **Small:** 1 backend developer (with access to product/UX inputs as needed).

Suggested Phases

Phase 1: Schema & Normalization Engine (Days 1–2)

- Key Deliverables: Request_scores table (all discrete columns), weight_config with full history, dynamic min-max normalization engine, JCR quartile mapping logic.
- Dependencies: pi_enrichment_cache and REDCap record schema finalized.

Phase 2: Subscore Calculation & Bonus Logic (Days 2–3)

- Key Deliverables: Category subscore calculation, NULL/partial handling, H5 redistribution, Niche Journal bonus multiplier via admin_config.
- Dependencies: Finalized admin_config entries.

Phase 3: Weighted Return, Effort, Rescaling (Days 3–4)

- Key Deliverables: Weighted Return Score, effort multiplier logic (including request-type fallback/default), dynamic queue-level ROI rescaling.
- Dependencies: Trac integration/fields ready.

Phase 4: Triggers & Weight Config (Days 4–5)

- Key Deliverables: Implement all rescore triggers, Apply logic, version incrementing, audit log; live preview as in-memory operation (no DB write).
- Dependencies: Config panel UI stub ready for integration.

Phase 5: QA & Validation (Days 5–7)

- Key Deliverables: Latency and reproducibility tests; NULL/zero validation across pipeline, audit fields populated, dynamic rescaling rank ordering confirmed.
- Dependencies: Product signoff and sample data.

Pending: IT & Compliance Question

"We are building an internal tool that stores REDCap research request metadata (PI name, department, study title, IRB number, funding source) and team hour logs from Trac. No patient data is stored. What is the required data retention period for these records, and are there any access control requirements specific to storage of IRB numbers in an internal application?"

Developer Callouts

1. All pipeline functions must distinguish explicitly between NULL (unknown/missing) and zero (known none)—never treat NULL as zero for normalization or scoring.
2. All category weights must be read from the active weight_config in DB; never hardcoded.
3. Every scoring event must log full input, normalized, intermediate, and output values for ML readiness.
4. Rescore triggers must write a new scoring event—never overwrite—preserving auditability.
5. QA test suite must validate correct flagging/color-coding for PARTIAL, COMPLETE, and FAILED statuses in score breakdown.
6. Out-of-bounds or negative metric values (e.g., negative grant amounts) must throw error and generate admin alert.
7. All overrides/manual notes are versioned and stored with user/timestamp.
8. H5 proxy redistribution logic and niche bonus logic are requirements, not options—exceptions must be logged.
9. Proxy scores must never be treated as equivalent to enrichment scores. proxy_confidence_discount (default 0.85) must be applied to all proxy-based subscores before entering the Weighted Return Score — read from admin_config, never hardcoded.
10. NULL faculty rank and NULL funding source = Tier 3 (insufficient data), not zero-scored. Zero means known low impact. NULL means unknown. This distinction must be preserved throughout the entire scoring pipeline.